1 Discuss the features of multiprocessor systems.

Ans:

A multiprocessor system consists of two or more CPUs that work together to solve a problem.

Features:

1. **Increased Throughput** : Multiple processors execute tasks in parallel, increasing the number of jobs completed per unit time.
2. **Economy** : More cost-effective than using several separate computers as the processors share memory, I/O devices and other resources.
3. **Reliability** : If one processor fails, others can continue to operate, increasing system reliability.
4. **Resource Sharing** : Processors share common memory and I/O devices, allowing easy access to data and communication.
5. **Scalability** : Performance can be improved by adding more processors to the system.
6. **Load Sharing** : Workload is distributed among processors to balance the load and reduce response time.
7. **Parallel Processing** : Different parts of a program can be executed simultaneously for faster execution.
8. **High Performance** : Better overall system performance for compute-intensive applications.
9. **Flexibility** : Multiprocessor systems can be designed using either tightly coupled (shared memory) or loosely coupled (distributed memory) architectures.
10. **Fault Tolerance** : System continues operation even if a processor or part of the system fails.

Types of Multiprocessor Systems :

1. Shared Memory Multiprocessor : All processors share a common memory.
2. Distributed Memory Multiprocessor : Each processor has its own local memory and communicates via an interconnection network.

2 Explain interconnection structures in multiprocessor systems.

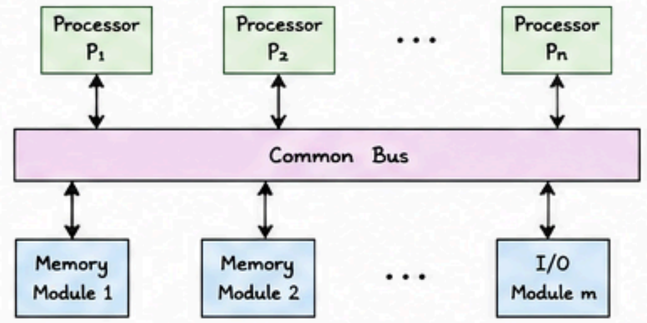
Ans:

Interconnection structure is the communication network that connects the processors, memory modules and I/O modules in a multiprocessor system. It provides paths for data transfer among system components.

Common Interconnection Structures:

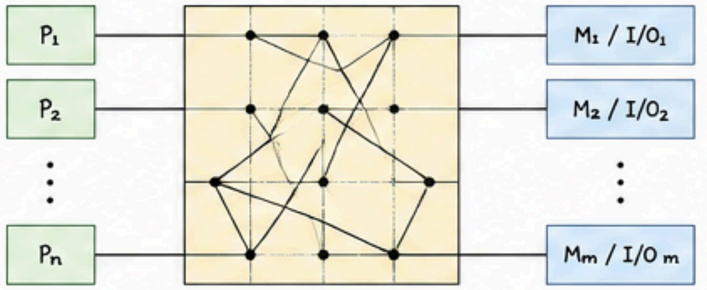
1. Bus (Common Bus)

- All processors, memory modules and I/O modules are connected to a single shared bus.
- Simple and low cost.
- Limited scalability due to bus contention.



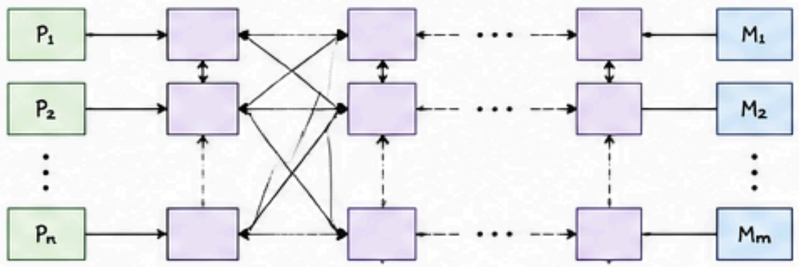
2. Crossbar Switch

- Each processor (and memory/I/O) is connected to every other module through a crossbar switch.
- Allows multiple simultaneous communications.
- High cost and complex.



3. Multistage Interconnection Network

- Uses a number of stages of switches to connect processors and memory modules.
- Examples: Omega network, Butterfly network, Banyan network, etc.
- Scalable and more practical for large systems.



Note :

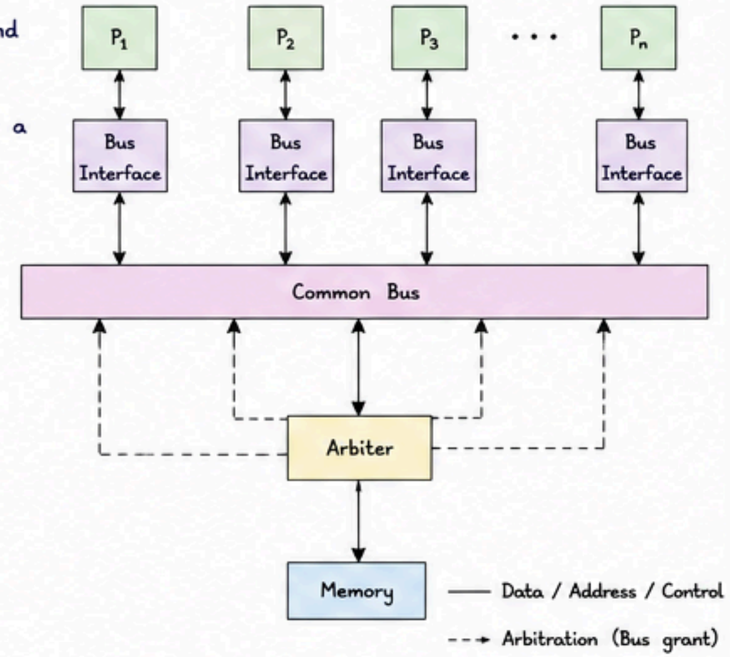
Selection of interconnection structure depends on performance, scalability, cost and complexity of the system.

3 Illustrate the time-shared common bus and multiport memory organization.

Ans:

(a) Time-shared Common Bus Organization

- All processors share a common bus and memory through an arbiter.
- Only one processor can use the bus at a time.
- The arbiter grants bus control to one requesting processor in turn.

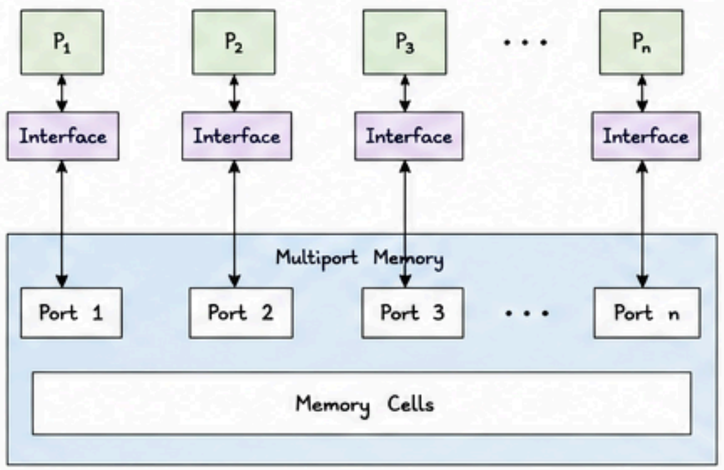


Operation:

- When a processor needs the bus, it sends a request to the arbiter.
- Arbiter selects one processor and grants the bus.
- The selected processor uses the bus to access memory or I/O.
- After the transfer, the bus is released and another processor can be granted.

(b) Multiport Memory Organization

- Memory has multiple ports so that several processors can access memory simultaneously.
- Each processor has a private path (port) to memory.
- Eliminates bus contention and increases parallelism.
- More expensive than time-shared bus organization.



Note:

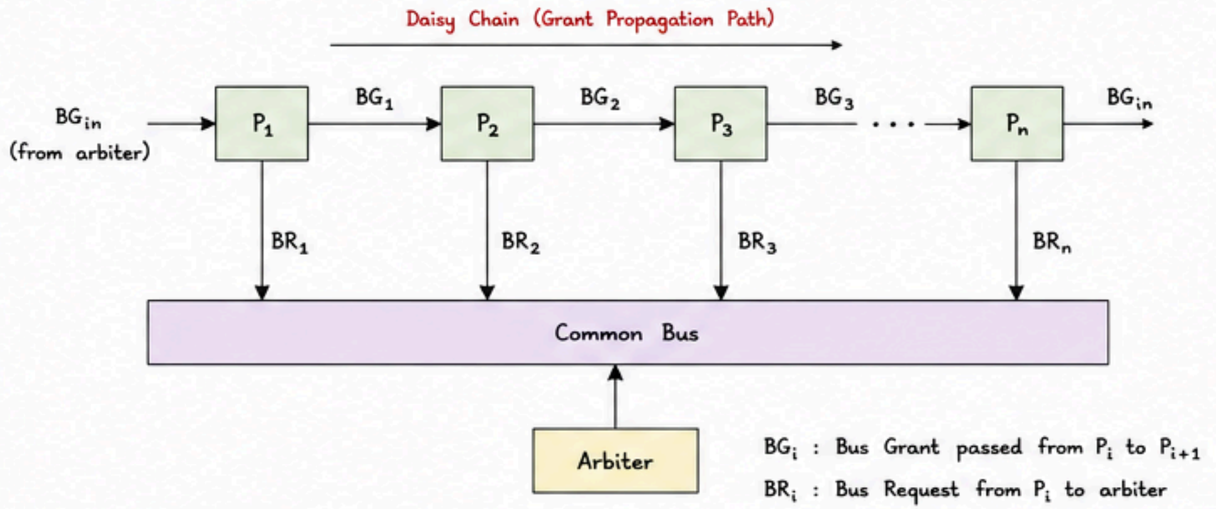
- Time-shared common bus is simple and economical but allows only one memory access at a time.
- Multiport memory allows multiple simultaneous memory accesses and gives higher performance.

4 Explain the serial (daisy chain) arbitration technique with a neat diagram.

Ans:

In serial (daisy chain) arbitration, a single grant signal is passed from one processor to the next in a fixed order. The first requesting processor that receives the grant captures the bus and blocks the grant from going further.

Diagram:



Working:

1. The arbiter issues a bus grant (BG_{in}) to the first processor (P_1).
2. The grant is passed serially from P_1 to P_2 to P_3 ... to P_n.
3. Any processor that has a bus request ($BR_i = 1$) captures the grant by not passing it further.
4. That processor gets control of the common bus and performs the transfer.
5. After completing the transfer, it releases the bus ($BR_i = 0$) and passes the grant to the next processor.
6. If no processor requests the bus, the grant eventually returns to the arbiter.

Advantages:

- Simple hardware.
- Requires only one grant line.
- Low cost.

Disadvantages:

- Fixed priority (P_1 has highest priority).
- More delay for processors at the end of the chain.
- Not fair for heavy traffic at the beginning.

Note:

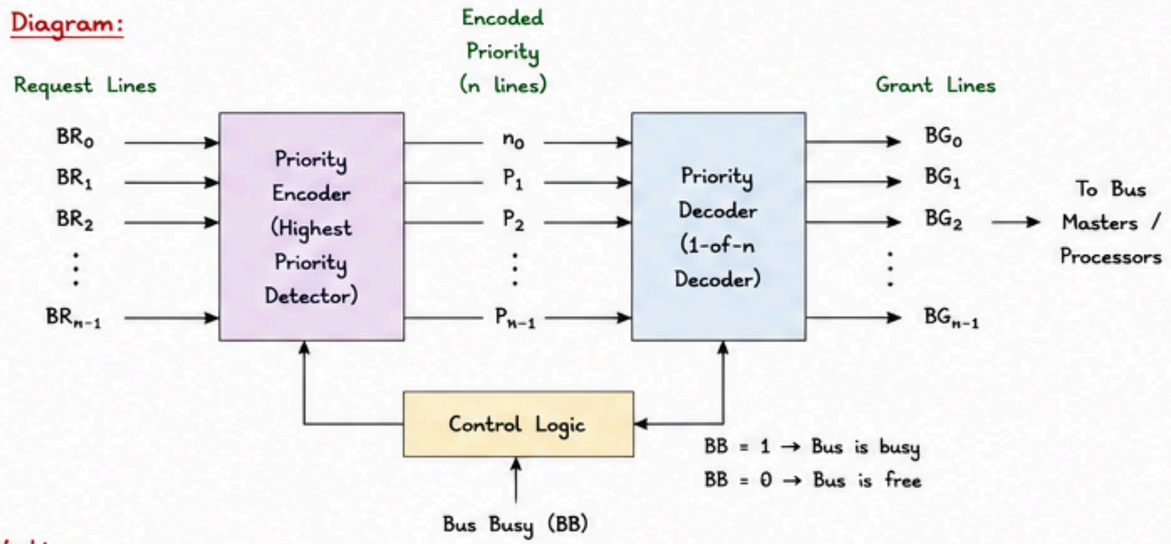
- The order of priority is fixed by the physical order of processors in the chain.
- It is widely used due to its simplicity despite the disadvantages.

5 Explain the parallel arbitration technique using priority encoder and decoder.

Ans:

In parallel arbitration, all requesting processors (or bus masters) send their requests simultaneously to the arbiter. The arbiter determines the highest priority requester using a priority encoder and then enables that one using a priority decoder.

Diagram:



Working:

1. All processors that need the bus assert their request lines ($BR_i = 1$) simultaneously.
2. The priority encoder examines all requests and produces an n-bit code corresponding to the highest priority requester.
3. The priority decoder receives this code and activates only one grant line ($BG_i = 1$) corresponding to that requester (one-hot output).
4. The granted processor uses the bus to perform its transfer.
5. After completion, it releases the bus (BB = 0), and the next round of arbitration begins.

Priority Example:

BR_3	BR_2	BR_1	BR_0	(Requests)	Encoder Output (P_1 P_0)	Grant Lines (BG_3 BG_2 BG_1 BG_0)
0	0	0	0	—	00	0 0 0 0
1	0	0	0	P_3	11	1 0 0 0
0	1	0	0	P_2	10	0 1 0 0
0	1	1	0	P_2	10	0 1 0 0
1	1	1	1	P_3	11	1 0 0 0

Advantages:

- Very fast since all requests are considered in parallel.
- Suitable for systems with many processors.
- Easy to expand.

Disadvantages:

- Requires more hardware (encoder + decoder).
- More wiring and cost compared to serial method.

Note:

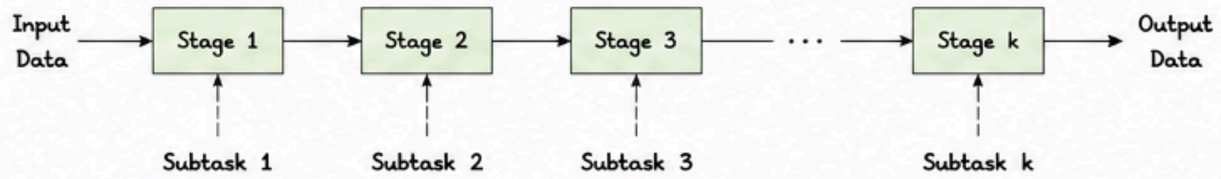
- Fixed priorities are assigned to request lines (e.g., BR_3 highest, BR_0 lowest). The encoder always selects the highest priority among the active requests.

6 Explain pipelining concept with examples.

Ans:

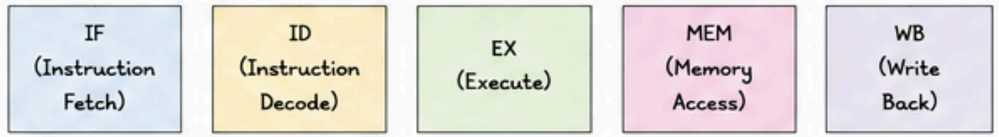
Pipelining is a technique in which a task is divided into a number of subtasks (stages) and these stages are arranged in a pipeline. Different subtasks are executed concurrently on different data items. While one data item is in a particular stage, other data items are in other stages. This increases the throughput and improves performance.

Illustration (k-stage pipeline):



Example 1: Pipelined Instruction Execution (5-Stage Instruction Pipeline)

An instruction is broken into 5 stages:



Multiple instructions are processed in an overlapped manner.

Pipeline Timing Diagram (for 5 instructions):

Clock Cycle → / Instruction	1	2	3	4	5	6	7	8	9
I_1	IF	ID	EX	MEM	WB	-	-	-	-
I_2	-	IF	ID	EX	MEM	WB	-	-	-
I_3	-	-	IF	ID	EX	MEM	WB	-	-
I_4	-	-	-	IF	ID	EX	MEM	WB	-
I_5	-	-	-	-	IF	ID	EX	MEM	WB

- In the first 5 cycles, the pipeline is being filled.
- From cycle 5 onwards, one instruction completes every clock cycle.
- For n instructions and k stages, total cycles = (n + k - 1).
- Throughput (after pipeline filled) = 1 instruction per cycle.

Advantages:

- Increases throughput.
- Better utilization of hardware.
- Higher performance.
- Reduces average response time.

Disadvantages:

- More hardware and control complexity.
- Hazards (data, control, structural) may occur.
- Pipeline latency increases.

Note:

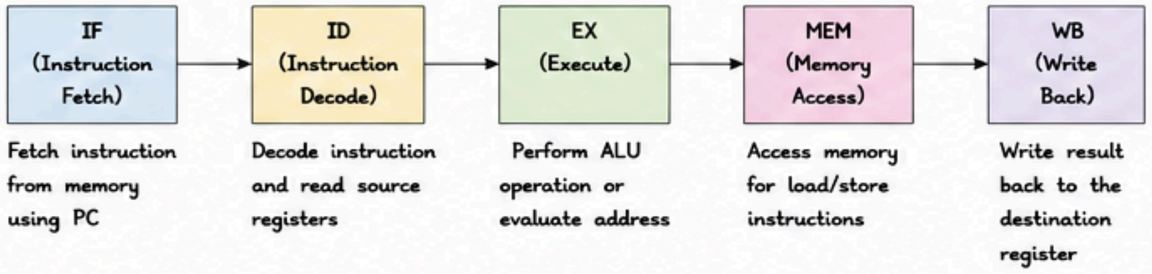
- Pipelining improves throughput, not the execution time of a single task. It allows overlapping of tasks similar to an assembly line.

7 Describe the instruction pipeline with its stages and explain with an example.

Ans:

An instruction pipeline is a set of hardware stages through which an instruction passes from fetch to completion. Different instructions are in different stages at the same time, allowing overlapped execution and higher throughput.

Pipeline Stages (5-Stage Instruction Pipeline)



Example : Consider the following 5 instructions

I_1: ADD R1, R2, R3
I_2: SUB R4, R1, R5
I_3: LW R6, 0(R1)
I_4: AND R7, R6, R8
I_5: OR R9, R1, R10

Pipeline Timing Diagram

Clock Cycle →	1	2	3	4	5	6	7	8	9
Instruction									
I_1	IF	ID	EX	MEM	WB	-	-	-	-
I_2	-	IF	ID	EX	MEM	WB	-	-	-
I_3	-	-	IF	ID	EX	MEM	WB	-	-
I_4	-	-	-	IF	ID	EX	MEM	WB	-
I_5	-	-	-	-	IF	ID	EX	MEM	WB

Observations:

- In the first 5 cycles, pipeline is being filled.
- From cycle 5 onwards, one instruction completes every clock cycle.
- After pipeline filled, throughput = 1 instruction per cycle.
- Total cycles for n instructions (k-stage pipeline) = n + k - 1. Here, n = 5 and k = 5 → Total cycles = 5 + 5 - 1 = 9.
- Pipelining increases throughput but not latency of a single instruction.

Note:

- Hazards (data, control, structural) may require stalls or forwarding.
- Ideal pipeline assumes no hazards.
- Modern processors use more than 5 stages for better performance.

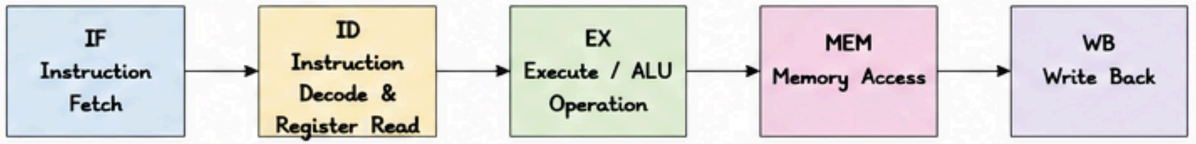
Summary : Instruction pipeline divides instruction execution into stages so that multiple instructions are processed simultaneauly in different stages, improving throughput and CPU utilization.

8 Explain the RISC pipeline and discuss its operation with an example.

Ans:

RISC (Reduced Instruction Set Computer) pipelines are designed to execute simple instructions in a small number of stages with fixed instruction length. Most RISC processors use a 5-stage pipeline.

RISC Pipeline (5-Stage)



Operation

1. Every clock cycle, each stage processes a different instruction.
2. After the first 5 cycles, a new instruction completes in every cycle.
3. RISC instructions are simple and usually complete in one EX stage.
4. Conditional branches may cause a control hazard; pipeline is flushed or stalled accordingly.

Example

Consider the following RISC instructions:

I_1: ADD R1, R2, R3
I_2: SUB R4, R1, R5
I_3: LW R6, 0(R1)
I_4: SW R6, 4(R1)
I_5: BEQ R1, R4, Label

Pipeline Timing Diagram

Clock Cycle →	1	2	3	4	5	6	7	8	9
Instruction									
I_1	IF	ID	EX	MEM	WB	-	-	-	-
I_2	-	IF	ID	EX	MEM	WB	-	-	-
I_3	-	-	IF	ID	EX	MEM	WB	-	-
I_4	-	-	-	IF	ID	EX	MEM	WB	-
I_5	-	-	-	-	IF	ID	EX	MEM	WB

Features of RISC Pipeline

- Simple instructions → short and uniform pipeline.
- Fixed instruction length → easy fetch and decode.
- Load/store architecture → only load/store access memory.
- Large number of registers → fewer memory accesses.
- High throughput and good performance.

Remarks

- Data hazards are handled using forwarding and stalls.
- Control hazards are handled using branch prediction or pipeline flush.

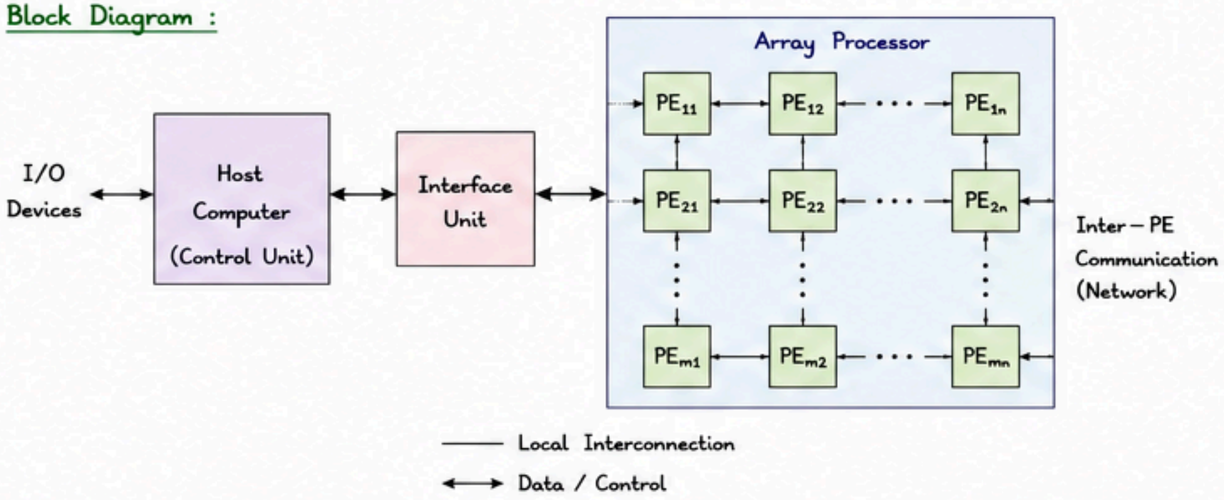
Summary : RISC pipeline uses a small number of simple stages to execute instructions. After the pipeline is filled, the CPU completes one instruction every clock cycle, achieving high throughput.

9 Describe the attached array processor using suitable diagram.

Ans:

An attached array processor (AAP) is a type of parallel processor in which a number of simple processing elements (PEs) are connected to a host computer und operate under its control. The host handles I/O, program control and communication; the array performs the intensive numerical computations in parallel.

Block Diagram :



Operation :

1. The host computer executes the main program, performs I/O and sets up data for the array.
2. Data and commands are sent to the array through the interface unit.
3. PEs execute the computations in parallel and exchange data through the interconnection network.
4. Results are returned to the host through the interface unit.

Features :

- Very high degree of parallelism.
- PEs are simple and inexpensive.
- Scalable by increasing the number of PEs.
- Suitable for scientific and engineering computations (e.g., matrix operations).
- Controlled and coordinated by a host computer.

Advantages :

- High processing power at low cost.
- Easy to design using simple PEs.
- Good for problems with regular, repetitive computations.
- Flexible – array size can be varied.

Limitations :

- Communication overhead in large arrays.
- Not efficient for irregular or non-parallel tasks.
- Performance depends on interconnection network.

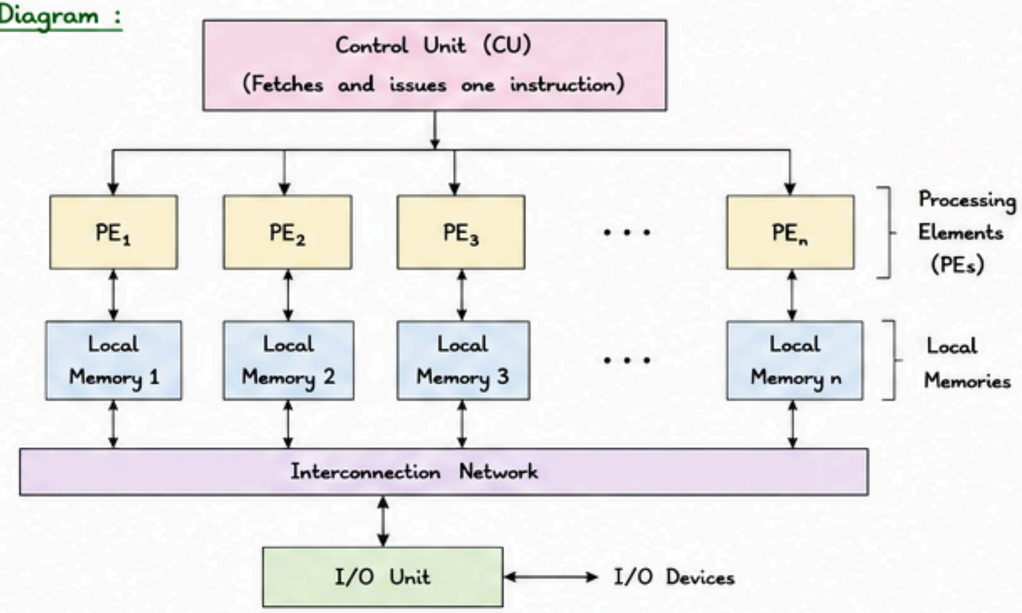
Summary : An attached array processor consists of an array of simple PEs connected to a host computer through an interface. It provides high parallelism and is well suited for numerical applications with uniform computations.

10 Describe the organization of a SIMD array processor with a neat diagram.

Ans:

A SIMD (Single Instruction, Multiple Data) array processor is a parallel processor in which a single control unit (CU) fetches and issues the same instruction to all processing elements (PEs) simultaneously. Each PE operates on different data elements.

Block Diagram :



Organization and Working :

1. The CU fetches one instruction from its memory.
2. The same instruction is broadcast to all PEs simultaneously.
3. Each PE performs the operation on its own local data.
4. PEs may exchange data through the interconnection network if required.
5. Results can be stored back to local memories or sent to I/O.

Characteristics :

- Single instruction stream.
- Multiple data streams.
- Synchronous execution.
- Each PE has its own local memory.
- High degree of parallelism.

Advantages :

- Simple control – only one instruction stream.
- High data throughput.
- Efficient for data-parallel applications (e.g., matrix operations, vector computations, image processing).
- Good scalability.

Limitations :

- All PEs execute the same instruction; not suitable for divergent control flow.
- Limited flexibility compared to MIMD systems.
- Performance depends on interconnection network and memory bandwidth.

Summary : A SIMD array processor has one control unit that issues the same instruction to all processing elements. Each PE works on different data in its local memory, providing highly parallel execution for data-parallel problems.